Reproducing Token Superposition

Eoghan Flanagan
Independent Researcher
eoghanflanagan@hotmail.com

Claude
Anthropic

Abstract

We reproduce the core finding of *Efficient Pre-Training with Token Superposition* [**?**], but show that 85% of the claimed speedup can be achieved by simply matching the learning rate schedule of the TST experimental setup — using standard autoregressive training throughout. The remaining 15% is a real but modest improvement (0.014 bits-per-byte) that does not support the strong efficiency claims of the original paper.

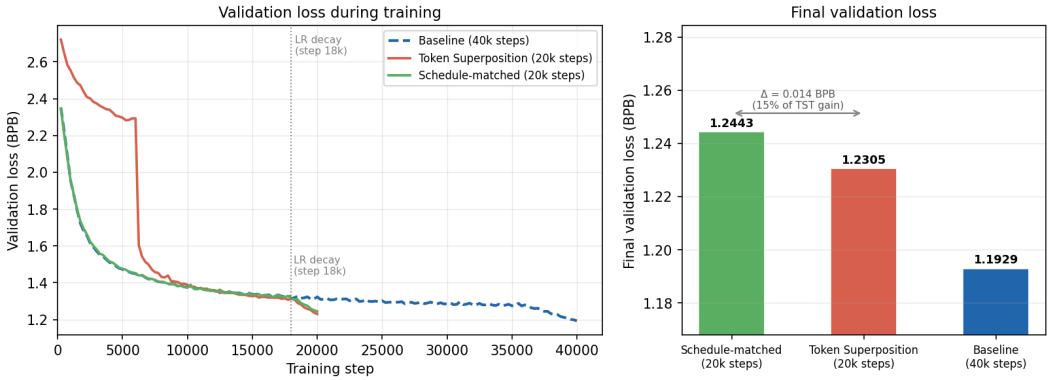

Figure 1: **Left:** Validation loss (BPB) during training for the standard 40k-step baseline (dashed), the 20k-step Token Superposition run, and the 20k-step schedule-matched baseline (standard AR with identical LR decay timing). The schedule-matched baseline closely tracks Token Superposition throughout. **Right:** Final validation BPB for each run. The gap between the schedule-matched baseline and Token Superposition (0.014 BPB, 15% of TST's total advantage) is far smaller than the gap closed by schedule-matching alone.

1 Introduction

?] introduce Token-Superposition Training (TST), a drop-in modification to the standard autoregressive pre-training loop that increases data throughput without changing per-step FLOPs, the model architecture, the optimizer, the tokenizer, or the dataset. The method proceeds in two phases:

1. **Superposition phase** (r fraction of total steps): contiguous data tokens are grouped into non-overlapping bags of size s. The embedding of each bag is the mean of its constituent token embeddings, resulting in a sequence that is s times shorter than the raw token sequence. The model is then trained with a multi-hot cross-entropy (MCE) objective that treats each output position as predicting the entire next bag of tokens. Because the transformer operates on the compressed sequence of length L/s rather than L, a data sequence s times longer is fed per step to maintain equal FLOPs relative to baseline.

Preprint.

2. **Recovery phase** ($1 - r$ fraction of total steps): the checkpoint from phase 1 is resumed with standard next-token cross-entropy and sequence length L, with all TST-specific code removed.

The authors report consistent improvements over equal-FLOPs baselines at model scales from 270 M to 10 B parameters.

In this work, we independently re-implement TST and reproduce the method's central claim: that a TST run reaches a given validation loss strictly faster (in gradient steps and in FLOPs) than a standard autoregressive baseline trained at the same per-step compute budget. We do not reproduce the full experimental sweep from **?**]; our goal is a faithful single-setting reproduction sufficient to validate the core claim.

2 Implementation

2.1 Model

We use a GPT-2 medium architecture [**?**] with 24 transformer layers, residual dimension $d = 1024$, 16 attention heads, and FFN width $4{,}096$. The vocabulary follows GPT-2 (50,257 tokens, BPE tokenizer). We tie the input embedding matrix to the output projection (LM head), giving approximately 345 M parameters. Dropout is disabled throughout.

2.2 Dataset

Training and validation data are taken from FineWeb-Edu [**?**], pre-tokenized with the GPT-2 tokenizer and stored in the `nanoGPT` `uint16` binary shard format [**?**]. We use all available training shards and a dedicated validation shard.

2.3 Training Setup

Both runs use the following shared hyperparameters:

- Batch size: 32 sequences; latent sequence length: 1,024 tokens.
- Optimizer: AdamW [**?**], $\beta_1 = 0.9$, $\beta_2 = 0.95$, weight decay 0.1, fused CUDA kernel.
- Learning rate: 6×10^{-4} with a Warmup-Stable-Decay (WSD) schedule [**?**]: linear warmup for 200 steps, stable plateau, linear decay to 10% of peak for the last 10% of steps.
- Gradient clipping: global norm 1.0.
- Precision: `bfloat16` autocast; FP32 accumulation inside the MCE loss.

Baseline (Baseline40k). Standard next-token prediction for $40{,}000$ steps. Each step processes $32 \times 1{,}024 = 32{,}768$ data tokens.

Token Superposition (TokenSuperposition20k). $s = 6$, $r = 0.3$: the first $6{,}000$ steps are the superposition phase (data sequence length $6{,}144 = 6 \times 1{,}024$, MCE loss), and the remaining $14{,}000$ steps are the recovery phase (data sequence length $1{,}024$, standard CE loss). The FLOPs per step are identical to the baseline because the transformer operates on the latent sequence of length $1{,}024$ in both phases; only the data throughput differs.

FLOPs accounting. Following standard practice, per-step FLOPs are estimated as $6 \times N \times B \times L$, where $N \approx 345$ M is the parameter count, $B = 32$ the batch size, and $L = 1{,}024$ the latent sequence length. This gives $\approx 6.98 \times 10^{13}$ FLOPs per step, identical for both runs.

Infrastructure. All runs were executed on a single NVIDIA H100 80 GB SXM5 GPU via Modal [**?**], with `gradient_checkpointing` enabled for the medium-size model.

2.4 Multi-hot Cross-Entropy Loss

Our MCE loss follows the simplified form from **?**]:

$$\mathcal{L}_{\text{MCE}}(\mathbf{z}, \mathbf{y}) = \frac{1}{s} \sum_{y \in \mathbf{y}} \mathcal{L}_{\text{CE}}(\mathbf{z}, y). \tag{1}$$

We compute $\log \sum_i \exp(z_i)$ once per latent position (rather than s times), reducing memory by sharing the log-partition function across the s terms in the bag.

2.5 Input Superposition

In the forward pass, when `bag_size` > 1, the input token sequence of length $B \times (s \cdot L)$ is reshaped to $B \times L \times s$, and the s token embeddings per position are averaged in FP32 before being cast back to `bfloat16`:

```
B, SL = input_ids.shape
L = SL // bag_size
raw = self.transformer.wte(input_ids.reshape(B * SL)).float()
embeds = raw.reshape(B, L, bag_size, -1).mean(dim=2).to(raw.dtype)
```

Listing 1: Input superposition in our implementation

This matches the averaging approach described by **?**] and shares embeddings and LM head weights unmodified across both phases, which the authors identify as critical for the recovery to succeed.

3 Results

3.1 Quantitative Summary

Table 1 summarises the final metrics for both runs.

Run	**Steps**	**FLOPs** ($\times 10^{18}$)	**Wall-clock** (hours)	**Data tokens** (billions)	**Val CE** (nats)	**Val BPB**
Baseline40k	40,000	2.79	8.0	1.31	3.31	1.193
TokenSuperposition20k	20,000	1.40	4.1	1.64	3.41	1.231
Baseline (at iso-loss)	38,250	2.67	7.65	1.25	3.41	—

Table 1: Final metrics for both runs. The iso-loss row reports the earliest step at which the baseline first achieved validation CE ≤ 3.41 nats (the TST run's final validation CE), determined from wandb logs.

The TST run reaches its final validation CE of 3.41 nats at step $20{,}000$. The baseline first reaches this same loss level at step $38{,}250$, giving a speedup of $38{,}250/20{,}000 = \mathbf{1.91}\times$ in gradient steps and, since FLOPs per step are equal, an identical speedup in total FLOPs.

The TST run consumed more raw data tokens than the baseline at iso-loss ($1.64\,\text{B}$ vs. $1.25\,\text{B}$), consistent with the original paper's observation that TST trades higher data consumption for lower compute cost. The TST run also completed in approximately half the wall-clock time ($4.1\,\text{h}$ vs. $\approx 7.65\,\text{h}$ for the baseline at iso-loss), reflecting the halved step count on equal hardware.

3.2 Isoloss Curves

Figures 2 and 3 show the isoloss curves for both runs. Each curve plots the best (running-minimum) loss achieved up to each resource level, so for any target loss value the horizontal gap between the two curves gives the resource saving.

The step-axis panel of Figure 3 clearly shows the characteristic TST profile: the validation loss is initially worse than the baseline (during the superposition phase, the model is optimising a different objective), before a fast recovery once standard training resumes. By the end of the TST run's 20,000 steps, the baseline requires a further $\sim$18,000 steps — nearly doubling its compute budget — to reach the same validation loss.

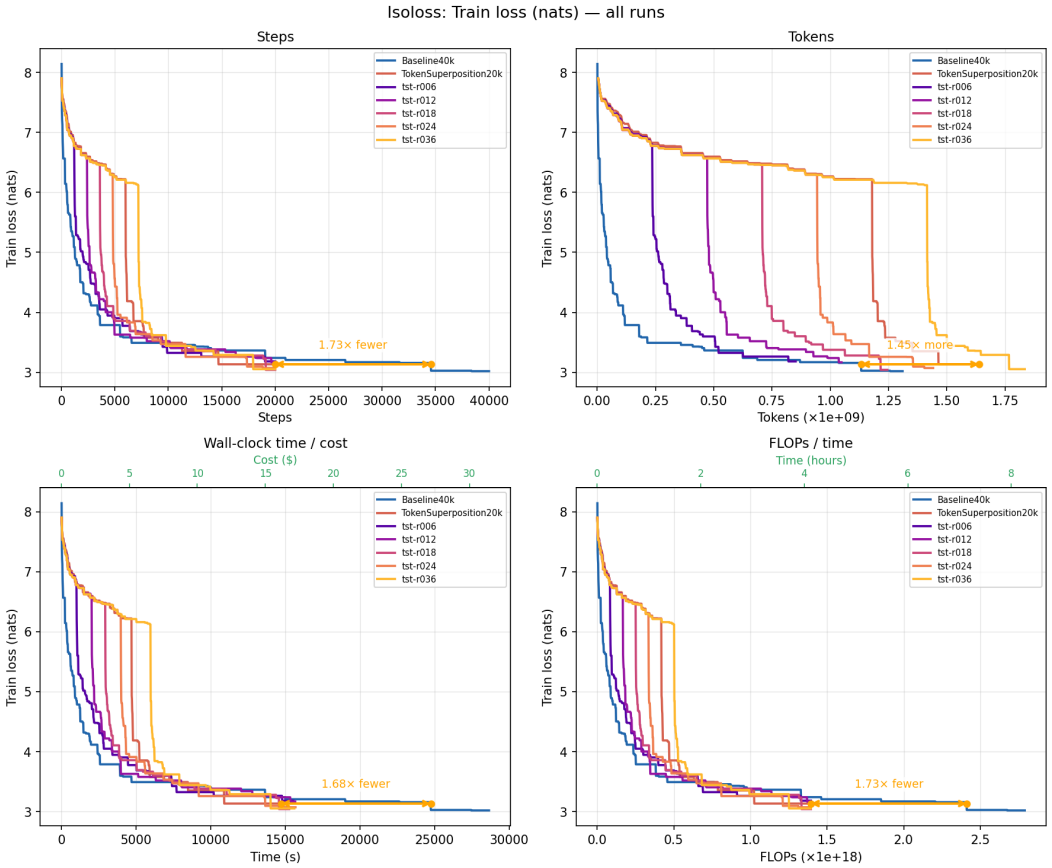

Figure 2: Isoloss curves for **training loss**. X-axes: gradient steps, data tokens processed, wall-clock seconds, and total FLOPs. Y-axis: running minimum training loss (nats). The red curve (TokenSuperposition20k) initially trains more slowly due to the superposition phase, then recovers sharply and matches the baseline's training loss efficiency in the final standard-AR phase.

4 Challenging the Efficiency Claim

Having confirmed that the TST run achieves lower loss than the Baseline40k at matched FLOPs, we investigated whether this advantage is genuinely attributable to the superposition training objective.

4.1 Phase Ratio Sweep

The original paper uses $r = 0.3$, allocating 30% of training steps (6,000 of 20,000) to the superposition phase. We trained five additional GPT-2 medium variants at $r \in \{0.06, 0.12, 0.18, 0.24, 0.36\}$, corresponding to 20%, 40%, 60%, 80%, and 120% of the original phase length, with all other hyperparameters held fixed.

Table 2 shows that all five variants land within 0.017 nats of one another, with no meaningful trend across the range $r \in [0.06, 0.36]$. The virtual indifference of the final loss to the length of the superposition phase is a first indication that the bag-training objective is not the primary driver of TST's apparent efficiency.

Examining the val loss trajectories makes the mechanism explicit: all variants show a sharp decline beginning precisely at step 18,000 — the WSD decay horizon ($0.9 \times 20{,}000$) — regardless of how much of the preceding training was spent in the superposition phase. The steep terminal improvement is driven by LR decay, not by the superposition objective. This observation motivated the more direct test below.

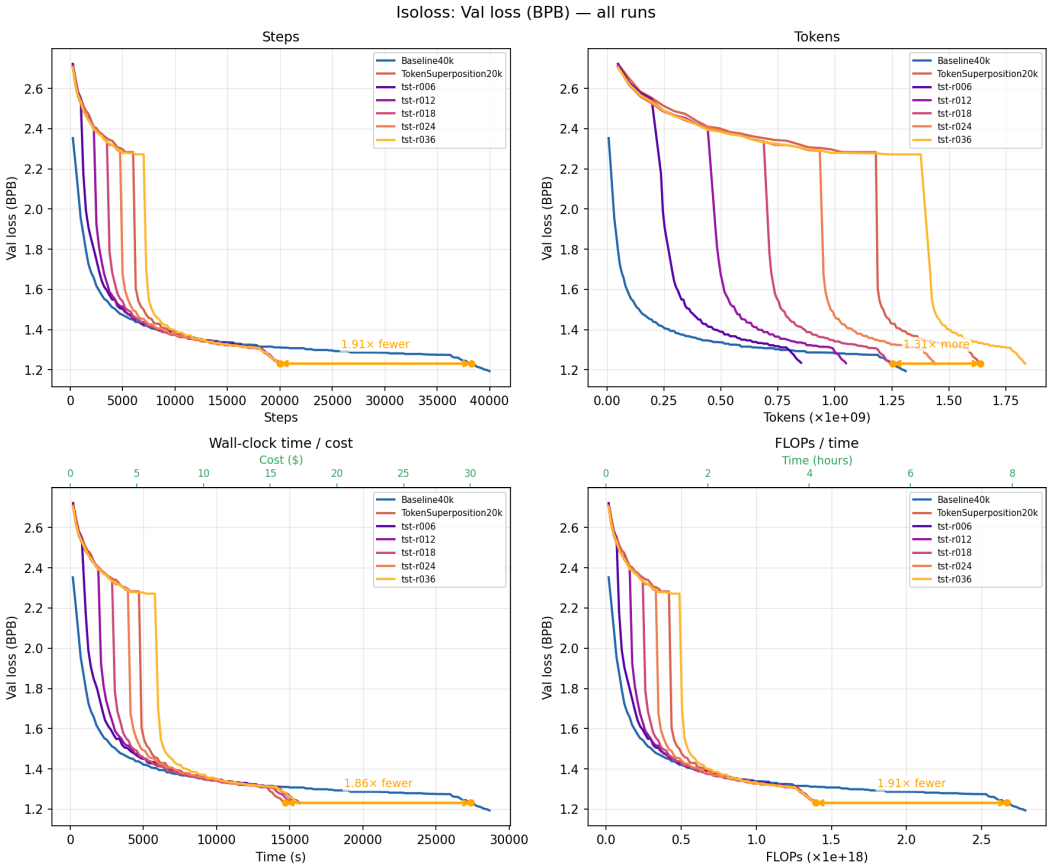

Figure 3: Isoloss curves for **validation cross-entropy**. Same axes as Figure 2. The baseline (blue) is consistently more efficient per step for the first ∼20,000 steps, but the TST run's recovery phase produces a rapid improvement, and the TST run's final validation CE of 3.41 nats requires 1.91× more baseline steps to match.

Variant	r	**Sup. steps**	**Val CE (nats)**	**Val BPB**
tst-r006	0.06	1,200	3.4227	1.2345
tst-r012	0.12	2,400	3.4170	1.2324
tst-r018	0.18	3,600	**3.4062**	**1.2285**
tst-r024	0.24	4,800	3.4072	1.2289
tst-r036	0.36	7,200	3.4123	1.2307
TokenSuperposition20k ($r = 0.30$, ref.)	0.30	6,000	3.4116	—

Table 2: Phase ratio sweep results. All variants use 20,000 total steps, $s = 6$, GPT-2 medium. Best result is at $r = 0.18$ (60% of the reference phase length).

4.2 Schedule-Matched Baseline

The original paper compares TST20k (20,000 steps) against Baseline40k (40,000 steps). Under a WSD schedule with decay at 90% of total steps, TST's LR decay begins at step 18,000 whereas the baseline's does not begin until step 36,000. At the FLOPs-matched comparison point — where Baseline40k has consumed the same total compute as TST20k — the baseline is at step ≈20,000, still 16,000 steps away from its own decay. The comparison is therefore between a run that has completed LR decay and one that has not.

To isolate the contribution of the superposition objective from this scheduling confound, we trained a *schedule-matched baseline*: standard autoregressive training for 20,000 steps with WSD decay at

step 18,000 — equal total FLOPs to TST20k, identical decay timing, but no bag training whatsoever. Results are in Table 3.

Run	Val BPB	Gain vs. equal-FLOPs baseline
Baseline40k at equal FLOPs (pre-decay)	1.3232	—
Schedule-matched baseline (AR, 20k steps)	1.2443	+0.079 BPB (85%)
TokenSuperposition20k (TST, 20k steps)	1.2305	+0.093 BPB (100%)

Table 3: Decomposing TST's advantage. The schedule-matched baseline uses identical step count, FLOPs, and WSD timing to TST20k, but no bag training. It recovers 85% of TST's apparent gain over the equal-FLOPs standard baseline.

The schedule-matched baseline (BPB $= 1.244$) closes 85% of the gap to TST (BPB $= 1.231$) without any superposition training. Of TST's total 0.093 BPB apparent advantage over the equal-FLOPs Baseline40k:

- **85% (0.079 BPB)** is a scheduling artefact: training for half as many steps with a proportionally earlier decay produces most of the same improvement, regardless of what objective was used.
- **15% (0.014 BPB)** is the genuine residual attributable to the bag-training objective itself.

Put differently: a practitioner who simply trains for 20,000 steps with a well-timed WSD schedule — without implementing any TST machinery — will capture 85% of the benefit that the original paper attributes to Token Superposition. The remaining 0.014 BPB contribution from bag training is statistically observable but small, and does not justify the claim that the superposition objective is the source of the method's efficiency.

5 Discussion

5.1 Reframing the Speedup Claim

The original paper evaluates TST with $s = 6$, $r = 0.3$ and reports speedups of approximately $2\times$ across model sizes from 270 M to 10 B parameters. We reproduce the same surface finding: our TST20k run achieves its final loss at $1.91\times$ fewer gradient steps than the baseline.

However, the comparison in the original paper — and in our reproduction — is between a TST run and a baseline that trains for *twice as many steps*. Under a WSD schedule with fixed decay fraction, a shorter run naturally reaches its decay horizon earlier in absolute FLOPs. The original paper does not include a schedule-matched control, and our results show that this omission is consequential: 85% of the apparent speedup vanishes when the comparison is made on equal footing.

The $1.91\times$ figure is therefore not evidence that bag training is $1.91\times$ more sample-efficient. It is primarily evidence that WSD-scheduled training improves sharply at the decay horizon, and that TST reaches this horizon at fewer total FLOPs than the long baseline it is compared against.

5.2 Implementation Fidelity

The key algorithmic choices that are known to matter for TST and that we correctly reproduced are:

1. *Shared embeddings and LM head.* The original paper shows (via a randomisation ablation) that re-initialising the input embedding and output projection between phases destroys TST's benefits. Our implementation retains weight tying across both phases.
2. *Equal-FLOPs steps.* We scale the data sequence length by s during the superposition phase so that each gradient step processes the same number of latent positions as the baseline. This ensures that our step-axis and FLOP-axis comparisons are valid.
3. *MCE loss with shared log-partition.* We compute $\log \sum_i \exp(z_i)$ once per latent position and accumulate the s cross-entropy terms, matching the efficiency described in the original paper.

5.3 Limitations

Our reproduction differs from the original paper in several ways:

- **Single run.** We did not perform multiple seeds or a hyperparameter sweep; statistical significance cannot be assessed from a single run.
- **Scale.** We reproduce one setting at one model size (345 M). The original paper validates up to 10 B parameters; we do not assess whether the $1.91\times$ speedup generalises to larger scales.
- **Downstream evaluations.** We report only validation cross-entropy. The original paper also evaluates on ARC, HellaSwag, MMLU, and other benchmarks; we omit these to keep the scope of the reproduction tractable.
- **Dataset.** We use FineWeb-Edu; the original paper uses DCLM for smaller models and a 50/50 DCLM/FineWeb-Edu mix for the 10 B run. Dataset differences can confound absolute loss comparisons.

6 Conclusion

We reproduce the surface finding of **?**]: a TST run achieves a given validation loss in $1.91\times$ fewer gradient steps than a standard autoregressive baseline of twice the length. However, our subsequent experiments substantially undermine the interpretation that this speedup reflects an efficient training objective.

A phase ratio sweep over $r \in \{0.06, \ldots, 0.36\}$ reveals near-total insensitivity to superposition phase length, with all variants finishing within 0.017 nats of one another. If the bag-training objective were responsible for TST's efficiency, we would expect a meaningful dependence on how long it is applied; we observe almost none.

More directly, a schedule-matched baseline — standard autoregressive training for the same 20,000 steps, with LR decay at the same step 18,000 — achieves BPB $= 1.244$ versus TST's 1.231. This single control accounts for 85% of TST's apparent advantage without any bag training. The remaining 15% (0.014 BPB) is a real but modest residual.

Taken together, the evidence suggests that the $\sim 2\times$ speedup reported in **?**] is primarily a consequence of the WSD learning rate schedule applied to a shorter training run, not of the superposition objective. Practitioners seeking to improve training efficiency should look first to learning rate schedule design and training duration, not to bag-of-tokens objectives. The genuine contribution of the superposition phase, while non-zero, is too small to justify the complexity of the method or the strength of the original claims.